ChatGPT

Podstawy Programowania 2025/2026 – Instrukcja projektu przygotowana przez Michała Małafiejskiego we współpracy z ChatGPT.

Totem Warrior's Race – tekstowa gra przygodowa (C, projekt dwufazowy)

Opis: Pilotowy projekt programistyczny łączący klasyczne programowanie w języku C z nowatorskim podejściem hybrydowego uczenia (human–AI). Projekt realizowany jest w dwóch fazach: najpierw student **samodzielnie implementuje od podstaw** podstawową wersję gry, a w drugiej fazie **rozbudowuje i udoskonala** projekt przy wsparciu narzędzi AI (np. ChatGPT, GitHub Copilot). Takie podejście ma umożliwić utrwalenie fundamentalnych umiejętności kodowania, a jednocześnie nauczyć krytycznego i efektywnego korzystania ze sztucznej inteligencji w programowaniu.

1. Opis projektu

Totem Warrior's Race to tekstowa gra przygodowa z elementami akcji, rozgrywana w konsoli. Gracz kieruje młodym wojownikiem plemienia, który uczestniczy w rytualnym wyścigu z totemem w ręku. Celem jest dostarczenie totemu na odległy ołtarz przed świtem, przemierzając niebezpieczną trasę naszpikowaną pułapkami i strzeżoną przez wrogów. Projekt zakłada stworzenie gry w trybie konsolowym (grafika tekstowa) z obsługą czasu rzeczywistego i implementację logiki gry od podstaw, a następnie stopniowe rozszerzanie jej funkcjonalności z pomocą AI.

2. Scenariusz gry

Słońce chyli się ku zachodowi nad pradawną doliną. Młody wojownik plemienia chwyta mocno swój święty totem i staje na linii startu rytualnego wyścigu. Przed nim rozciąga się naszpikowana pułapkami trasa – starożytne ruiny, zwodnicze bagna i gęsty las. Gdzieś w oddali czeka cel: ołtarz, na którym musi złożyć totem przed nadejściem świtu. Na drodze czekają go przeszkody i czyhający wrogowie – dzikie bestie oraz strażnicy totemu, którzy zrobią wszystko, by go powstrzymać. Wojownik musi zbierać rozproszone **totemy mocy** ukryte na trasie, dające mu siłę lub otwierające ukryte ścieżki. Czas ucieka nieubłaganie, a każdy błąd może być ostatnim. Czy zdoła przechytrzyć niebezpieczeństwa i dostarczyć totem na czas, zapewniając swojemu ludowi przychylność duchów?

Cel gry: Pokieruj wojownikiem tak, aby – omijając przeszkody i wrogów – zebrać wszystkie totemy na planszy i dotrzeć do mety przed upływem czasu (lub zanim wyczerpią się życia bohatera). Tylko szybki refleks, sprytne manewry i mądre korzystanie z mocy totemów pozwolą wygrać ten wyścig.

3. Cele projektu

Projekt umożliwi Ci zdobycie i przećwiczenie następujących umiejętności:

- **Programowanie w C:** Utrwalenie podstaw języka C – praca z funkcjami, strukturami (`struct`), wskaźnikami, operacjami na plikach itp.

- **Grafika tekstowa i I/O w czasie rzeczywistym:** Opanowanie wyświetlania grafiki tekstowej oraz obsługi wejścia/wyjścia w trybie rzeczywistym (reagowanie na naciśnięcia klawiszy bez pauzowania programu).
- **Programowanie zdarzeniowe:** Zrozumienie i implementacja głównej pętli gry reagującej na zdarzenia, z kilkoma obiektami poruszającymi się jednocześnie.
- **Konfiguracja zewnętrzna:** Wykorzystanie zewnętrznego pliku konfiguracyjnego do definiowania parametrów rozgrywki, co uczy tworzenia elastycznych, konfigurowalnych aplikacji.
- **Refaktoryzacja i rozbudowa kodu:** Rozwinięcie umiejętności refaktoryzacji i optymalizacji kodu, a także dodawania nowych funkcjonalności do istniejącej aplikacji.
- **Współpraca z AI:** Nabycie doświadczenia we współpracy z AI przy programowaniu – formułowanie skutecznych zapytań (promptów), ocena odpowiedzi AI i integracja użytecznych sugestii do własnego kodu.

4. Wymagania technologiczne

- **Język i kompilator:** Implementacja gry w języku C (standard ANSI C99 lub nowszy). Możesz użyć dowolnego zgodnego kompilatora (np. GCC, Clang, MSVC).
- **Środowisko:** Aplikacja działa w trybie tekstowym (konsola/terminal) – nie przewiduje się interfejsu graficznego.
- **Biblioteki:** Dopuszczalne jest korzystanie ze standardowej biblioteki C (stdio, stdlib, time itp.) oraz ewentualnie biblioteki do grafiki tekstowej i obsługi klawiatury w konsoli: **ncurses** (Unix/Linux, macOS) lub alternatywnie <conio.h> (Windows). Niedozwolone jest użycie zaawansowanych zewnętrznych bibliotek graficznych ani silników gier – gra ma pozostać prostą grą konsolową.
- **Uruchamianie:** Gra powinna uruchamiać się w terminalu i reagować na wejście z klawiatury **bez zatrzymywania pętli gry**. Należy zastosować nieblokujące odczyty znaków (np. tryb `nodelay` w bibliotece ncurses, funkcje `kbhit()` / `getch()` z <conio.h> lub podobne mechanizmy), aby program mógł jednocześnie odświeżać stan gry. Rozgrywka toczy się w czasie rzeczywistym – główna pętla powinna odświeżać stan gry wiele razy na sekundę, stale sprawdzając wejście użytkownika i upływ czasu.

5. Ograniczenia projektowe

Podczas implementacji musisz przestrzegać następujących obowiązkowych ograniczeń:

- **Paradygmat proceduralny:** Stosuj programowanie strukturalne. Niedozwolone jest programowanie obiektowe (C++), w szczególności klasy, dziedziczenie, polimorfizm, przeciążanie operatorów itp. Kod pisz w czystym C (ewentualnie w C++ ograniczonym do stylu C). Możesz tworzyć własne struktury danych (`struct`) i funkcje, ale nie metody wewnątrz struktur (czyli żadnych klas/obiektów).
- **Biblioteki i struktury danych:** Zakazane jest użycie biblioteki szablonów C++ (STL) lub innych gotowych kontenerów. Wszystkie potrzebne struktury danych (listy, tablice, kolejki zdarzeń itp.) zaimplementuj samodzielnie lub użyj prostych struktur z C. Dopuszcza się korzystanie z funkcji biblioteki standardowej (np. do obsługi plików, generowania liczb losowych).
- **Zmienne globalne:** Nie używaj zmiennych globalnych (wyjątkiem są stałe proste typu, zdefiniowane `const`). Dopuszczalne jest globalne zdefiniowanie stałych konfiguracji (np. rozmiary planszy, symbole grafiki), jeśli to konieczne, ale wszystkie zmienne przechowujące stan gry (pozycje, liczniki, wskaźniki na obiekty) powinny być przekazywane między funkcjami przez parametry lub przechowywane w strukturach danych.
- **Rozmiar funkcji:** W celu wymuszenia czytelności i modularności kodu obowiązuje **limit maksymalnie ~1024 bajtów kodu na jedną funkcję** (dotyczy także `main`). Do tego rozmiaru

nie wliczają się białe znaki i komentarze. Innymi słowy, każda funkcja powinna być na tyle krótka, by jej kod źródłowy (pomijając spacje, nowe linie i komentarze) zmieścił się w około 1024 znakach. Jeśli jakaś funkcja robi się dłuższa – podziel ją na mniejsze, logiczne części.

- **Niezastosowanie się do powyższych ograniczeń:** Będzie traktowane jako niespełnienie wymagań projektu i może skutkować **niezaliczeniem (0 punktów)**, niezależnie od działania gry.

6. Zasady rozgrywki (mechanika gry)

Gra **Totem Warrior's Race** jest jednoosobową grą akcji w trybie tekstowym. Gracz kontroluje postać wojownika przemierzającego planszę złożoną z pól znakowych (siatka ASCII). Plansza może reprezentować trasę wyścigu lub labirynt pełen niebezpieczeństw, złożony z różnych symboli oznaczających teren, przeszkody, obiekty i postacie. Na przykład kropka (`.`) może oznaczać wolne pole, krzyżyk (`#`) ścianę/przeszkodę, symbol `T` totem do zebrania, a bohatera symbol `W` (lub inna wyróżniająca się ikona).

Cel rozgrywki to dotrzeć do wyznaczonego miejsca (mety) oraz zebrać po drodze wszystkie totemy rozmieszczone na planszy, zanim skończy się czas lub zanim bohater straci wszystkie życia. Każdy totem podniesiony przez gracza zwiększa jego wynik punktowy lub daje mu specjalną korzyść (np. chwilową ochronę – tę mechanikę można opcjonalnie rozszerzyć).

Przeciwnicy: W trakcie rozgrywki na planszy pojawiają się ruchomi przeciwnicy (wrogowie) sterowani przez komputer. Mogą to być np. strażnicy poruszający się po wyznaczonych ścieżkach, dzikie zwierzęta krążące tam i z powrotem albo inne niebezpieczeństwa. Kontakt z wrogiem jest groźny – w najprostszym wariancie natychmiast kończy grę (postać ginie), albo odbiera wojownikowi jedno życie i cofa go na start (jeśli implementujemy system wielu żyć). Gracz musi unikać kolizji z przeciwnikami, planując ruchy w odpowiednim tempie.

Przeszkody: Na planszy mogą znajdować się również przeszkody statyczne (`#` lub inne ustalone symbole) – np. głazy, drzewa, pułapki – których nie można przekroczyć. Stanowią one utrudnienie, zmuszając bohatera do obrania innej drogi do celu. *(Opcjonalnie, niektóre przeszkody mogą działać jak pułapki wyzwalające się po wejściu, ale w podstawowej wersji traktujemy je po prostu jako blokady terenu.)*

Sterowanie: Postać gracza porusza się na podstawie wejścia z klawiatury. Używaj klawiszy `W/A/S/D` (lub strzałek) do sterowania ruchem (standardowo: `W` = góra, `S` = dół, `A` = lewo, `D` = prawo). Ruch odbywa się w dyskretnych krokach o jedno pole na siatce. Przy każdej iteracji pętli gry program sprawdza, czy użytkownik nacisnął któryś z klawiszy, i odpowiednio przemieszcza bohatera, o ile ruch w danym kierunku jest dozwolony (bohater nie próbuje wyjść poza planszę ani wejść w przeszkodę). Jeśli ruch jest niedozwolony (np. próba wejścia w przeszkodę lub poza obszar planszy), postać pozostaje na miejscu – gra powinna nadal działać (ignorować niepoprawny ruch). Upewnij się, że odczyt klawiszy nie blokuje działania programu (np. zastosuj tryb `nodelay` w ncurses lub podobny mechanizm).

Widoczny obszar (scrolling): Plansza gry może być większa niż fragment widoczny jednocześnie na ekranie. Należy zaimplementować przesuwanie widoku (scrolling) wraz z ruchem bohatera, tak aby ekran wyświetlał tylko fragment większej planszy – najbliższe otoczenie postaci. To ważna cecha projektu: gracz nie powinien widzieć od razu całej mapy, a jedynie tę część, którą aktualnie eksploruje.

Czas i punkty: Rozgrywka jest ograniczona w czasie – np. uruchamiany jest licznik czasu (powiedzmy startowo ~60 sekund, lub inna wartość z pliku konfiguracyjnego). Upływający czas powinien być pokazywany na panelu statusu. Gracz wygrywa, jeśli zrealizuje cel (zbierze wszystkie totemy i dotrze do mety) przed upływem czasu. Jeśli czas się skończy lub bohater straci wszystkie życia zanim osiągnie cel,

następuje porażka. Po drodze gracz zdobywa punkty za zebrane totemy (i ewentualnie inne osiągnięcia); suma punktów również powinna być wyświetlana (np. w panelu statusu i/lub w komunikacie końcowym).

7. Zadania – Faza I (samodzielna implementacja podstawowej gry)

W pierwszej fazie Twoim zadaniem jest **samodzielne zaimplementowanie od podstaw** działającej, podstawowej wersji opisanej powyżej gry. Skup się na minimalnej funkcjonalności, spełniając poniższe wymagania. **Nie korzystaj z pomocy AI ani gotowych rozwiązań** – celem jest, byś własnoręcznie zmierzył się z problemami i stworzył rdzeń gry od zera. Wymagania **obowiązkowe** Fazy I (minimalny zakres, aby zaliczyć podstawę projektu) to:

1. **Interfejs tekstowy i główna pętla gry (1 pkt):** Stwórz tekstową **planszę gry** oraz **panel statusu** wyświetlane w konsoli. Plansza powinna przedstawiać obszar gry w formie siatki znaków (wymiary planszy mogą być zdefiniowane w pliku konfiguracyjnym). Panel statusu (np. u góry lub dołu ekranu) powinien pokazywać bieżące informacje o rozgrywce, m.in. **pozostały czas** do końca gry, **liczbę zebranych/pozostałych totemów**, aktualny **wynik punktowy**, a także ewentualnie liczbę żyć gracza czy aktualny poziom. Upewnij się, że ekran jest na bieżąco odświeżany w trakcie działania pętli gry i zawsze prezentuje aktualny stan rozgrywki. Zaimplementuj główną **pętlę zdarzeń**, która cyklicznie aktualizuje logikę gry oraz odświeża wyświetlaną planszę i panel (zgodnie z opisem mechaniki). Dla czytelności grafiki tekstowej ustal legendę znaków reprezentujących elementy gry (np. `W` – wojownik, `T` – totem, `#` – przeszkoda, `E` – przeciwnik, `.` – puste pole itp.). Jeżeli używana biblioteka na to pozwala, zastosuj także **kolory** do wyróżnienia ważnych elementów (np. bohater w jednym kolorze, wrogowie w innym, totemy w jeszcze innym).

2. **Sterowanie postacią i ruch (1 pkt):** Zaimplementuj sterowanie bohaterem za pomocą klawiatury. Postać powinna reagować na naciśnięcia klawiszy w każdej iteracji pętli gry – przesuwać się o jedno pole w żądanym kierunku (góra/dół/lewo/prawo) zgodnie z klawiszami `W/A/S/D` lub strzałkami. Należy uwzględnić ograniczenia ruchu: **nie pozwól wyjść poza obszar planszy ani wejść na pole z przeszkodą**. Jeżeli gracz próbuje wykonać ruch, który jest niemożliwy (np. w kierunku ściany `#`), program powinien po prostu zignorować ten ruch (postać pozostaje w miejscu, gra toczy się dalej). Upewnij się, że sposób odczytu klawiszy nie blokuje działania programu (np. w ncurses ustaw tryb bezblokujący `nodelay`, w Windows skorzystaj z `kbhit()` itp.), tak aby pętla gry mogła działać płynnie niezależnie od tempa naciskania klawiszy. Ruch postaci powinien być na tyle płynny, na ile pozwala tryb tekstowy – gracz powinien móc szybko reagować na zagrożenia.

3. **Totemy i przeszkody (1 pkt):** Rozmieść na planszy kilka **totemów** do zebrania przez gracza oraz **przeszkody** statyczne (np. symbol `#`). Po wejściu bohatera na pole z totemem należy: zwiększyć licznik **zebranych totemów** (oraz ewentualnie dodać punkty do wyniku) i usunąć ten totem z planszy (zamienić na znak pustego pola). *(Możesz opcjonalnie dodać od razu jakiś efekt dla gracza po zebraniu totemu – np. +1 życie, spowolnienie czasu – ale nie jest to wymagane w fazie I.)* Przeszkody (`#`) ustaw zgodnie z ustalonym układem planszy (np. wczytanym z pliku) i traktuj jako pola **nieprzekraczalne** – ani postać gracza, ani przeciwnicy nie mogą przez nie przechodzić. Upewnij się, że logika kolizji rozróżnia, czy dany obiekt na polu to totem (wtedy należy go podnieść) czy przeszkoda (nie można na nią wejść). Wszystkie totemy rozmieszczone na mapie **muszą zostać zebrane**, aby gracz mógł osiągnąć cel gry – możesz np. w panelu statusu na bieżąco wyświetlać, ile totemów pozostało do zebrania.

4. **Przeciwnicy – ruch i kolizje (1 pkt):** Wprowadź do gry co najmniej jeden typ **wroga** sterowanego przez komputer. Może to być np. przeciwnik poruszający się poziomo tam i z powrotem między dwoma punktami, poruszający się losowo w wyznaczonym obszarze albo strażnik patrolujący korytarz. (Dla uproszczenia implementacji możesz zacząć od ruchu losowego lub prostego cyklicznego – strategię zachowania wroga wybierz według uznania.) Ruch wrogów powinien być obsługiwany w pętli gry automatycznie, z pewną ustaloną prędkością – np. wróg wykonuje jeden krok co kilka iteracji pętli, żeby nie był zbyt szybki w porównaniu z graczem. Zadbaj o to, by przeciwnicy również respektowali granice planszy i nie przenikali przez przeszkody (np. powinni zawracać lub „odbijać się" od ścian, albo zmieniać kierunek zgodnie z ustalonym wzorcem ruchu). **Kolizje z graczem:** Jeśli postać gracza znajdzie się na tym samym polu co przeciwnik (czy to gracz wejdzie na pole wroga, czy wróg na pole gracza), uznaj to za kolizję. W podstawowej wersji gry kolizja powinna skutkować **utratą życia przez gracza lub natychmiastowym końcem gry**. Najprostsze rozwiązanie: gdy dojdzie do spotkania z wrogiem – gra się kończy (bohater ginie). Można też zaimplementować system żyć: np. gracz ma kilka żyć, traci jedno przy kolizji, a jeśli ma jeszcze pozostałe życie to kontynuuje (np. startując od pozycji startowej), a jeśli było to ostatnie życie – koniec gry. *(Wybór podejścia pozostawiamy Tobie, system żyć nie jest konieczny w fazie I.)*

5. **Ograniczenie czasu gry i warunki zakończenia (1 pkt):** Zaimplementuj **odliczanie czasu** i warunki wygranej/przegranej związane z upływem czasu oraz zrealizowaniem celu. Na starcie rozgrywki ustaw określony **limit czasu** (np. 60 sekund, albo wczytany z pliku konfiguracyjnego) i wyświetlaj aktualny pozostały czas w panelu statusu. Zegar powinien **odliczać w dół** w trakcie gry. Jeżeli czas się wyczerpie zanim gracz zbierze wszystkie totemy i dotrze do mety, rozgrywka kończy się porażką (timeout). Analogicznie, jeżeli gracz straci wszystkie życia (o ile wprowadzono system żyć) przed osiągnięciem celu, gra również kończy się przegraną. Z kolei gdy graczowi uda się zebrać wszystkie wymagane totemy **i** dotrzeć do pola mety przed upływem czasu, gra kończy się **wygraną**. Po zakończeniu rozgrywki (zarówno wygranej, jak i przegranej) wyświetl odpowiedni **komunikat** podsumowujący wynik – np. *"Wygrana! Zdobyłeś 5/5 totemów w 47 sekund."* lub *"Koniec gry – czas upłynął."*. Następnie możesz zaoferować możliwość rozpoczęcia gry od nowa lub zakończenia programu (opcjonalnie).

6. **System punktacji (1 pkt):** Dodaj zliczanie **punktów** i prezentację wyniku. Najprościej – za każdy zebrany totem przyznaj graczowi pewną liczbę punktów (np. +100) i sumuj punkty w trakcie rozgrywki. Możesz także przyznawać punkty za inne osiągnięcia (np. bonus za pozostały czas, punkty za pokonanie poziomu itp.), ale w fazie I wystarczy podstawowy system. Bieżący wynik punktowy wyświetlaj na panelu statusu. Po zakończeniu gry (wygranej lub przegranej) pokaż końcową liczbę punktów gracza (np. w komunikacie końcowym). *(Zapisywanie rankingu najlepszych wyników nie jest wymagane w fazie I – to będzie ewentualne rozszerzenie w fazie II.)*

Uwagi do fazy I: Powyższe punkty obejmują minimalne wymagania funkcjonalne. Zadbaj również o poprawność i **stabilność działania** podstawowej gry – przetestuj ją w różnych scenariuszach (wygrana, przegrana, różne układy planszy z pliku konfiguracyjnego). Faza I ma pokazać Twój **samodzielny wkład** w projekt, dlatego cały kod powinien być Twojego autorstwa i w pełni przez Ciebie zrozumiany. Staraj się od początku pisać kod w sposób czytelny i dobrze zorganizowany (sensowne nazwy zmiennych i funkcji, unikanie powtórzeń, podział na moduły zgodnie z potrzebą, komentarze tam gdzie trzeba, przestrzeganie ograniczeń z sekcji 5). Choć jakość kodu nie jest osobno punktowana w fazie I, ułatwi Ci to pracę w fazie II i będzie tam oceniana.

8. Zadania – Faza II (rozszerzenie i refaktoryzacja z pomocą AI)

W drugiej fazie projektu będziesz korzystać z AI jako asystenta, aby **ulepszyć swoją grę**. Po ukończeniu Fazy I (działającej podstawowej wersji gry) możesz zaangażować narzędzia AI (np. ChatGPT, GitHub Copilot) do analizy i rozbudowy Twojego kodu. Pamiętaj jednak, że nadal **Ty jesteś autorem** – AI ma Ci pomóc, ale to Ty decydujesz, co ostatecznie trafi do projektu. Wykorzystaj tę fazę, aby **zrefaktoryzować kod** (uczynić go lepszym jakościowo) oraz dodać **nowe funkcjonalności**, czyniąc grę ciekawszą i bardziej dopracowaną. Kluczowym elementem jest tu prowadzenie **dziennika interakcji z AI** – dokumentowania, o co prosisz AI, jakie otrzymujesz sugestie i jak je oceniasz (szczegóły w sekcji *Portfolio projektu*).

Poniżej przedstawiono listę możliwych usprawnień i rozszerzeń gry, które możesz wprowadzić w Fazie II. Nie musisz realizować wszystkich – **wybierz te, które uznasz za interesujące** lub które AI pomoże Ci zrealizować. Im więcej sensownych rozszerzeń zaimplementujesz (przy jednoczesnym zachowaniu wysokiej jakości kodu), tym wyższą ocenę możesz zdobyć. Traktuj tę listę jako inspirację i dodawaj też **własne pomysły** – kreatywność jest mile widziana!

1. Refaktoryzacja kodu i optymalizacje (zalecane)

- Poproś AI o **przegląd kodu** z Fazy I pod kątem czytelności, błędów i potencjalnych ulepszeń. Zaimplementuj sensowne sugestie, takie jak:
- lepszy podział programu na moduły i funkcje (np. wydzielenie osobnych funkcji typu `aktualizujPrzeciwników()`, `sprawdzKolizje()`, itp., jeśli do tej pory cały kod jest w jednej pętli),
- usunięcie powtarzającego się kodu, poprawa nazewnictwa zmiennych dla większej klarowności, dodanie komentarzy w złożonych fragmentach,
- optymalizacje wydajnościowe – np. ulepszenie algorytmu sprawdzania kolizji (unikanie zbędnych pętli), zmniejszenie zużycia pamięci (alokowanie tylko potrzebnych struktur) itp.,
- wykrycie i poprawa błędów lub obsługa pominiętych sytuacji brzegowych – AI może pomóc zauważyć np. brak sprawdzania wyniku `malloc`, możliwość wyjścia poza tablicę, nieprawidłową inicjalizację zmiennej, itp. Napraw te kwestie,
- sprawdzenie zgodności z ograniczeniami projektu – upewnij się, że żadna funkcja nie przekracza dozwolonej długości (AI może podpowiedzieć, jak rozbić zbyt duże funkcje na mniejsze),
- ogólne ulepszenia jakości: np. uporządkowanie kodu tak, aby łatwiej było dodawać nowe funkcje w przyszłości.
- *Pamiętaj, by krytycznie oceniać propozycje AI – nie wszystkie będą trafne. W dzienniku interakcji opisz, które sugestie były użyteczne, a które odrzuciłeś (i dlaczego).*

2. Nowe elementy gry (rozszerzenia funkcjonalne)

Dodaj **nowe funkcjonalności** wykraczające poza podstawowe wymagania z Fazy I. Wymyśl je samodzielnie lub skorzystaj z podpowiedzi AI. Poniżej pogrupowano propozycje rozszerzeń tematycznie:

- **System punktacji i ranking:** Rozbuduj system punktów – np. przyznawaj punkty za każdy zebrany totem (+100 pkt), bonus za pozostały czas, dodatkowe punkty za ukończenie poziomu. Możesz także zapisywać **ranking najlepszych wyników** do pliku (np. `wyniki.txt`) i wyświetlać go po ukończeniu gry lub przy starcie (np. pokazując top 5 wyników).
- **Wiele poziomów i rosnący poziom trudności:** Po ukończeniu pierwszej planszy gra może automatycznie przechodzić do kolejnego **poziomu**. Zaprojektuj kilka poziomów (np. w osobnych plikach konfiguracyjnych) o rosnącym stopniu trudności: więcej przeszkód, szybsi lub liczniejsi przeciwnicy, krótszy limit czasu. AI może pomóc wygenerować pomysły na układy plansz. Możesz

również zaimplementować dynamiczne zwiększanie trudności na jednym poziomie – np. z upływem czasu przeciwnicy przyspieszają albo pojawiają się nowi wrogowie.

- **Nowi przeciwnicy o zróżnicowanej inteligencji:** Dodaj różne typy wrogów. Na przykład:
- *Łucznik* – przeciwnik strzelający pociskami (reprezentowany innym symbolem, pociski poruszają się w linii prostej).
- *Patrol* – przeciwnik chodzący po określonej trasie (np. krąży wokół przeszkody lub strzeże danego obszaru).
- *Łowca* – wróg podążający za graczem: co pewien czas namierza położenie gracza i kroczy w jego kierunku (możesz poprosić AI o prosty algorytm pogoni).
- *Boss* – specjalny przeciwnik na finał poziomu, z bardziej złożonymi zachowaniami (np. porusza się szybciej, może przenikać przez przeszkody lub okresowo zmienia tryb ataku).
  (Wprowadzenie nowych wrogów wiąże się z dodaniem nowych reguł kolizji – np. co się dzieje po trafieniu pociskiem – oraz być może nowych symboli graficznych i parametrów do konfiguracji, takich jak częstotliwość strzałów łucznika. Zastanów się, jak to obsłużyć w kodzie.)
- **Przyjazne elementy i power-upy:** Dodaj obiekty, które pomagają graczowi:
- *Sojusznik* – np. przyjazne zwierzę albo inny wojownik, który po dotknięciu przez gracza dołącza na krótko i eskortuje go przez kilka pól, czyniąc gracza tymczasowo nietykalnym.
- *Power-upy* – przedmioty do podniesienia inne niż totemy, dające graczowi tymczasowe moce: np. Tarcza (chwilowa nietykalność przez kilka sekund), Buty szybkości (przyspieszenie ruchu gracza), Zamrożenie czasu (chwilowe wstrzymanie ruchu wszystkich wrogów). *(Możesz zapytać AI o pomysły i wskazówki implementacyjne takich mechanik.)*
- **Elementy środowiskowe:** Uatrakcyjnij świat gry poprzez dodatkowe mechanizmy otoczenia:
- *Pułapki aktywne* – np. zapadnie (`^`), które aktywują się z opóźnieniem po nadepnięciu (gracz ma ułamek sekundy na ucieczkę), albo kolce wysuwające się rytmicznie co pewien czas.
- *Ruchome przeszkody* – np. pływające platformy na rzece, ruchome piaski itp. (wymaga to dodatkowej logiki ruchu tych obiektów, ale AI może zasugerować rozwiązanie).
- *Zmienne warunki (pogoda, cykl dnia i nocy)* – np. co pewien czas zmienia się widoczność w grze (w trybie tekstowym można to zasymulować zmianą kolorów tła/przeciwników, choć tu możliwości są ograniczone).
- **Ulepszony interfejs i grafika tekstowa:** Z pomocą AI dodaj elementy upiększające grę:
- Kolorowe **animacje ASCII** – np. migający symbol totemu, animowana postać bohatera (zmieniająca co pewien czas znak, by symulować ruch), efekty przy zbieraniu totemu (krótki „błysk” koloru w konsoli).
- **Lepsze komunikaty** – wykorzystaj AI, by sformułować ciekawsze teksty dla gracza (np. komunikaty przy wygranej/przegranej, przy utracie życia, opis fabularny wprowadzający do kolejnego poziomu).
- **Dźwięk (opcjonalnie):** w konsoli nie ma łatwego dostępu do audio, ale możesz np. generować proste sygnały systemowe (`\a`) lub tworzyć „efekty dźwiękowe” z ASCII art (np. rysując *BUM!* po wybuchu pułapki).
- **Menu i opcje gry:** Dodaj na początku programu menu tekstowe z opcjami: *Start gry*, *Instrukcja*, *Wyniki*, *Zakończ*. (AI może podpowiedzieć implementację prostego menu w konsoli). W opcji instrukcja wyświetl skrótowo zasady sterowania i celu gry; w opcji wyniki – odczytaj i pokaż zawartość pliku z rankingiem najlepszych wyników.

Pamiętaj, aby wszelkie nowe parametry gry (np. liczba i rodzaj wrogów, czasy trwania power-upów, układy kolejnych poziomów itp.), o ile to możliwe, również wczytywać z pliku konfiguracyjnego (lub z osobnych plików dla kolejnych poziomów). Dzięki temu Twój projekt będzie bardziej modułowy i elastyczny.

3. Testowanie i poprawa niezawodności

Wykorzystaj AI do wygenerowania dodatkowych **scenariuszy testowych** i nietypowych sytuacji, które mogłyby sprawić problem Twojej grze. Przykładowe kwestie do sprawdzenia:

- **Kolizje wrogów:** Co się stanie, jeśli dwaj przeciwnicy zderzą się ze sobą albo spróbują wejść jednocześnie na to samo pole? Upewnij się, że gra to obsłuży (np. wrogowie mogą się „przenikać" bez efektu, albo możesz zapobiec takiej sytuacji odpowiednią konfiguracją ich tras).
- **Timer i koniec gry:** Czy licznik czasu nie zejdzie przypadkiem poniżej zera? Czy po upływie czasu gra na pewno kończy się i pokazuje właściwy komunikat? Czy po wygranej lub porażce **wszystkie pętle** zostają przerwane, a program czeka na decyzję gracza (restart/koniec) zamiast np. dalej odliczać czas?
- **Różne rozdzielczości konsoli:** Jak zachowa się gra przy różnych rozmiarach okna terminala (liczbie kolumn/wierszy)? *(Możesz poprosić AI o sugestie, jak np. dynamicznie dostosować wyświetlanie do rozmiaru okna lub wykryć wymiary terminala).*
- **Zarządzanie pamięcią:** Sprawdź (ręcznie lub z pomocą AI) czy Twój program nie ma wycieków pamięci – jeśli używasz `malloc`, upewnij się, że każda alokacja ma odpowiadające `free` w odpowiednim momencie. AI może pomóc w analizie kodu pod tym kątem.
- **Błędy w pliku konfiguracyjnym:** Zabezpiecz program na wypadek nieprawidłowego formatu danych wejściowych. Co jeśli w pliku konfiguracyjnym brakuje jakiejś wartości albo ma zły format? Dodaj komunikaty o błędach i sensowne wartości domyślne, lub w razie poważnego błędu w konfiguracji – bezpiecznie zakończ program z informacją o problemie zamiast doprowadzić do crashu (np. `segmentation fault`).

Poza powyższymi, pamiętaj że Faza II to **proces iteracyjny**: zapytaj AI o pomysły -> otrzymaj sugestie -> **zastosuj lub zmodyfikuj** -> przetestuj -> zapytaj ponownie... Dokumentuj te kroki w swoim dzienniku interakcji. Możesz np. w kolejnych iteracjach pytać: *„Przeanalizuj moją funkcję kolizji, czy widzisz błąd?"*, potem *„Zaproponuj nowe funkcje, które uatrakcyjnią grę."*, dalej *„Jak zaimplementować ranking zapisujący wyniki do pliku?"* i tak dalej. Staraj się korzystać z języka angielskiego w pytaniach do modeli (większość narzędzi AI lepiej rozumie precyzyjny techniczny angielski). **Każdą istotną zmianę** w kodzie dokładnie przetestuj samodzielnie – AI nie zawsze przewidzi wszystkie konsekwencje ani nie wychwyci błędów integracji.

Ważnym elementem Fazy II jest także Twoja umiejętność **oceny i krytyki** odpowiedzi AI. Nie przyjmuj bezrefleksyjnie każdej podpowiedzi – jeśli AI proponuje coś, co nie pasuje do Twojej wizji albo wydaje się błędne, masz prawo (a nawet obowiązek) to **odrzucić lub zmodyfikować**. W dokumentacji (dzienniku AI) zapisuj również takie sytuacje: np. *„AI zasugerowało zmianę X, ale ją pominąłem, ponieważ..."*. Taka refleksja pokaże, że potrafisz świadomie korzystać z pomocy AI, a nie tylko bezkrytycznie polegasz na jej podpowiedziach.

Podsumowując Fazę II: postaraj się wynieść swój projekt na wyższy poziom. Wykorzystaj moc AI do realizacji zadań trudnych lub żmudnych, ale dbaj o to, by **rozumieć kod**, który dodajesz z jej pomocą. Finalnie powinieneś dysponować znacznie ulepszoną grą (względem końca Fazy I) oraz bogatszym doświadczeniem we współpracy z inteligentnym asystentem programowania.

9. Zasady oceniania

Projekt będzie oceniany **holistycznie**, z uwzględnieniem zarówno końcowego produktu (działającej gry), jak i procesu jego powstawania (Twojej pracy i współpracy z AI).

Łącznie do zdobycia jest 18 punktów: 6 punktów za Fazę I + 12 punktów za Fazę II.

- **Faza I (6 pkt):** Oceniamy, czy podstawowa wersja gry spełnia wszystkie minimalne wymagania funkcjonalne z Fazy I i poprawnie działa. Za każdy z kluczowych elementów (interfejs i pętla, sterowanie, totemy, przeciwnicy, czas i warunki końca, punkty) przyznawany jest w przybliżeniu 1 punkt – niezaimplementowanie lub poważne błędy w danym aspekcie oznaczają utratę tej części punktów. Aby zaliczyć projekt, finalna gra (po Fazie II) **musi** realizować podstawowe założenia z Fazy I (gra powinna się uruchamiać, reagować na sterowanie, poprawnie obsługiwać mechaniki ruchu, kolizji, totemów, przeciwników, czasu itp.). Krytyczne błędy uniemożliwiające rozgrywkę lub brak któregoś z kluczowych elementów oznaczają niespełnienie wymagań projektu. Poprawne działanie podstawy jest **konieczne**, choć samo w sobie niewystarczające do najwyższej oceny.

- **Kryteria oceny Fazy II (łącznie 12 pkt):**

- **Poprawność działania i jakość kodu (4 pkt):** Oceniamy poprawność działania Twojej gry po rozbudowie oraz jakość jej kodu. Gra powinna być stabilna, zgodna z założeniami i wolna od poważnych błędów. Sprawdzamy czy finalna wersja spełnia wszystkie wymagania (także te z Fazy I) oraz czy nowe funkcje działają poprawnie. Równie istotna jest jakość kodu: jego czytelność, organizacja (modularność, sensowna struktura), przestrzeganie ograniczeń projektowych, komentarze w newralgicznych miejscach. Kod powinien być lepszy jakościowo dzięki refaktoryzacji w Fazie II. *(Uwaga: Jeśli finalna gra nie spełnia podstawowych wymagań z Fazy I, projekt może nie zostać zaliczony niezależnie od innych dokonań.)*
- **Rozszerzenia i kreatywność (4 pkt):** Oceniamy zakres i pomysłowość zaimplementowanych rozszerzeń w Fazie II. Czy dzięki pomocy AI udało Ci się **znacząco ulepszyć** projekt w porównaniu do wersji podstawowej? Dodatkowe funkcje powinny być sensowne, dobrze przemyślane i poprawnie zaimplementowane. Wyżej oceniane będą projekty, w których widać **własną inwencję** wspartą AI – np. nietypowy, ciekawy pomysł na mechanikę rozgrywki, interesujące urozmaicenie oprawy gry albo sprytne usprawnienia kodu. Pamiętaj, że musisz **rozumieć** wszystkie wprowadzone zmiany: jeśli AI zasugerowało użycie jakiejś struktury danych czy algorytmu, powinieneś umieć krótko wyjaśnić, jak to działa i dlaczego zostało zastosowane.
- **Zaangażowanie, wysiłek i refleksyjność (4 pkt):** Bierzemy pod uwagę cały **proces tworzenia** projektu, na podstawie dostarczonej dokumentacji. Z przeanalizowania Twojego *Dziennika realizacji (Faza I)* oraz *Dziennika interakcji z AI (Faza II)* będziemy wnioskować, ile pracy i iteracji włożyłeś w projekt. Student, który systematycznie pracował nad grą, napotykając problemy i stopniowo je rozwiązując, będzie miał bogatszy zapis działań (wiele wpisów z opisami prób i błędów, testów, poprawek). Taki zapis świadczy o realnym wysiłku i nauce podczas pracy nad projektem. Z kolei bardzo ubogi dziennik (albo jego brak) może sugerować, że projekt powstał „cudem" w ostatniej chwili lub kod został wygenerowany bezrefleksyjnie – co odbije się negatywnie na ocenie. Promujemy podejście, w którym **nauka wymaga wysiłku** – chcemy widzieć, że projekt kosztował Cię pracę intelektualną (co zresztą zwykle przekłada się na lepsze opanowanie materiału). Ważna jest także **refleksyjność** – w portfolio powinny znaleźć się Twoje przemyślenia na temat przebiegu projektu: co było trudne, czego się nauczyłeś, co byś zrobił inaczej następnym razem. Umiejętność wyciągania wniosków ze swoich doświadczeń (zarówno sukcesów, jak i błędów czy ograniczeń AI) jest bardzo cenna i również podlega ocenie.

10. Portfolio projektu (wymagane elementy dokumentacji)

Gotowy projekt oddajesz w formie tzw. **portfolio**, na które składają się kod źródłowy gry oraz krótka dokumentacja złożona z kilku sekcji (może to być plik README, PDF lub inny czytelny format). Celem

portfolio jest udokumentowanie zarówno **co** zrobiłeś, jak i **jak** do tego doszedłeś. Poniżej przedstawiono oczekiwaną strukturę sprawozdania oraz wskazówki do każdej części:

- **Opis gry i założeń początkowych:** Na początku raportu krótko opisz *własnymi słowami*, na czym polega Twoja gra. Przedstaw scenariusz (fabułę) i główne zasady: kim jest gracz, jaki ma cel, jakie elementy występują (totemy, wrogowie, przeszkody) i jakie są warunki wygranej/przegranej. Napisz, jakie **minimalne funkcjonalności** zaplanowałeś w Fazie I, a jakie **pomysły na rozszerzenia** miałeś w Fazie II (nawet jeśli nie wszystkie udało się zrealizować). Możesz tu także dołączyć np. szkic przykładowej planszy czy diagram zależności modułów, jeśli to pomoże zrozumieć strukturę projektu. Dodatkowo podaj, z jakiego środowiska korzystałeś do rozwoju (system operacyjny, kompilator, IDE) **oraz potwierdź, że w Fazie I korzystałeś jedynie z dozwolonych źródeł** (materiały kursowe, dokumentacja, własne notatki – *bez użycia AI ani gotowych rozwiązań z Internetu*).

- **Zmiany i nowe funkcje po Fazie II:** W tej sekcji wymień **konkretnie** zmiany i ulepszenia, które wprowadziłeś do gry w Fazie II w porównaniu z początkową wersją z Fazy I. Wypunktuj każde rozszerzenie lub poprawkę i krótko opisz jej charakter. Na przykład:

- *Dodano ranking graczy:* Gra zapisuje najlepsze wyniki do pliku `wyniki.txt` i po zakończeniu rozgrywki wyświetla tabelę Top 5.
- *Nowy przeciwnik – Łucznik:* przeciwnik strzelający pociskami, które zadają obrażenia z dystansu. Zaimplementowano pociski poruszające się po prostej.
- *Refaktoryzacja pętli gry:* główna pętla została podzielona na funkcje `aktualizujObiekty()`, `sprawdzKolizje()` itp., co poprawiło czytelność.

Postaraj się krótko opisać **każdą istotną zmianę** – tak, aby osoba sprawdzająca wiedziała, na co zwrócić uwagę testując Twoją grę i przeglądając kod.

- **Dziennik realizacji – Faza I (praca samodzielna):** Jest to chronologiczny zapis Twoich prac nad podstawową wersją gry (Faza I). Podziel go na etapy lub dni – zgodnie z tym, jak przebiegała Twoja praca. Każdy wpis powinien zawierać **datę lub numer etapu** oraz opis: nad czym pracowałeś, jakie napotkałeś problemy i jak je rozwiązałeś. Pisz zwięźle i konkretnie, koncentrując się na najważniejszych krokach. Przykład wpisu:

  Dzień 1: Rozpocząłem od zaimplementowania wczytywania mapy z pliku i wyświetlenia jej w konsoli. Natrafiłem na problem z funkcją `fscanf` – błędnie odczytywała znaki nowej linii, przez co układ planszy się psuł. Rozwiązałem to, używając `fgets` zamiast `fscanf` do odczytu linii tekstu.

  Dzień 2: Zaimplementowałem sterowanie postacią w pętli gry. Początkowo postać poruszała się z opóźnieniem – okazało się, że `getch()` z biblioteki ncurses domyślnie blokuje program czekając na input. Znalazłem w dokumentacji tryb `nodelay`, ustawiłem go i teraz pętla działa płynnie. Przetestowałem ruch i kolizje ze ścianami – działają poprawnie.

Staraj się odnotować **kluczowe momenty**: gdy trafiłeś na trudny błąd, jak go debugowałeś; gdy dodałeś nową funkcję i jak ją przetestowałeś; itp. Ten dziennik ma pokazać, **jak myślałeś i uczyłeś się** w trakcie samodzielnego kodowania. Jeśli testowałeś grę na bieżąco, możesz wspomnieć o wynikach testów (np. *"Sprawdziłem, że limit czasu działa – po odliczeniu do zera gra kończy się komunikatem o porażce."*).

Uwaga: Ten element portfolio jest dowodem Twojej samodzielnej pracy przed użyciem AI. Dziennik powinien być napisany w czasie przeszłym i pierwszej osobie liczby pojedynczej („zrobiłem…", „napotkałem…", „nauczyłem się…").

- **Dziennik interakcji z AI – Faza II:** Ta część dokumentuje Twoją współpracę z AI w drugiej fazie, krok po kroku. Również prowadzona chronologicznie – obejmuje **ważniejsze sesje z AI** podczas ulepszania gry. Dla każdej istotnej interakcji zanotuj:
- **Pytanie/Problem:** co zapytałeś lub jak sformułowałeś prośbę do AI (możesz streścić zamiast podawać pełny prompt, jeśli był długi).
- **Odpowiedź AI:** jaką odpowiedź lub sugestię otrzymałeś (również skrótowo).
- **Działanie:** co zrobiłeś z otrzymaną sugestią – czy wdrożyłeś ją (dosłownie lub z modyfikacjami), czy też postanowiłeś ją odrzucić.
- **Ocena:** krótka ocena, na ile ta podpowiedź okazała się użyteczna – czy pomogła rozwiązać problem, usprawnić kod, czy może była nietrafiona.

Przykład wpisu w dzienniku AI:

> *Pytanie:* Zapytałem ChatGPT, jak poprawić wykrywanie kolizji, bo czasem gra nie wykrywała zderzenia z wrogiem (wkleiłem fragment kodu pętli).
> *Sugestia AI:* Model przeanalizował kod i zasugerował, by sprawdzać kolizje dopiero po zaktualizowaniu pozycji wszystkich obiektów, a nie w trakcie iterowania listy wrogów.
> *Działanie:* Porada okazała się trafna – przerobiłem kod zgodnie z nią i problem zniknął.
> *Ocena:* Sugestia była bardzo przydatna, AI pomogło znaleźć błąd, którego sam wcześniej nie zauważyłem.
>
> *Pytanie:* Poprosiłem AI o pomysł na nową mechanikę – zaproponowało dodanie pułapek środowiskowych.
> *Sugestia AI:* Odpowiedź była dość ogólna, ale nakierowała mnie na pomysł z kolcami wysuwającymi się co kilka sekund.
> *Działanie:* Wprowadziłem mechanikę kolców z drobnymi własnymi zmianami (losowe opóźnienie wysunięcia).
> *Ocena:* Pomysł AI okazał się inspirujący – sam bym na to nie wpadł, choć implementacja wymagała doprecyzowania.
>
> *Pytanie:* Spytałem GitHub Copilota o pomoc w optymalizacji rysowania ekranu.
> *Sugestia AI:* Zasugerował użycie bufora off-screen zamiast wypisywania znak po znaku na bieżąco.
> *Działanie:* Wdrożyłem to – dodałem bufor znaków i po każdej iteracji wypisuję go jednym poleceniem na ekran.
> *Ocena:* Rzeczywiście zauważyłem poprawę wydajności. Sugestia bardzo przydatna.

Pamiętaj: nie oczekujemy pełnych logów czatu – skup się na esencji: czego dotyczyło pytanie, jaka była rada AI, jak ją wykorzystałeś i z jakim rezultatem. Szczególnie istotne jest odnotowanie przypadków, gdy AI myliło się lub zaproponowało coś nieadekwatnego i musiałeś z tego zrezygnować – to pokaże Twoje krytyczne podejście. Np. AI podało kod funkcji w C++, choć projekt miał być w C, więc musiałem go przerobić; poza tym zaproponowało użycie biblioteki X, która nie była dozwolona.

- **Podsumowanie (refleksje końcowe):** Na koniec dokumentacji napisz swoje **przemyślenia** o projekcie. Zastanów się: co było dla Ciebie trudne, czego nowego się nauczyłeś, co byś zrobił inaczej następnym razem? Możesz ocenić, w czym AI najbardziej pomogła, a gdzie musiałeś

samodzielnie dopracować rozwiązanie. Taka szczera autorefleksja jest bardzo cenna – pokazuje Twoją zdolność do wyciągania wniosków z doświadczenia.
